

## Project Card

### Cat's Odyssey

---

#### Project:

Cat's Odyssey

#### Purpose:

Videojuego 2D de plataformas y puzzles para PC, protagonizado por Preciosa. El jugador recorre escenarios en pixel art que acompañan una historia de tono oscuro, misterioso y emocional sobre el regreso con su dueña. La jugabilidad combina desplazamiento libre, salto, adherencia/trepado, empuje de cajas, evasión de enemigos, coleccionables y checkpoints. La arquitectura debe permitir agregar y ajustar niveles, enemigos y mecánicas sin duplicar innecesariamente la lógica.

## Stakeholder Card

### Usuario / Jugador

---

#### Project:

Cat's Odyssey

#### Stakeholder:

Usuario / Jugador

#### Goal:

Comprender rápidamente controles y reglas, resolver plataformas y puzzles con señales claras y disfrutar la historia sin frustración causada por comportamientos inconsistentes.

#### Quality Attributes:

Learnability (Usability) - QA-Priority: 3

Las acciones básicas deben comprenderse durante los primeros minutos mediante controles y señales consistentes.

Error tolerance / Recovery (Usability) - QA-Priority: 2

Al morir o equivocarse, el jugador debe comprender qué ocurrió y retomar el desafío desde un estado predecible.

Satisfaction (Usability) - QA-Priority: 2

La dificultad, el feedback y el ritmo deben evitar frustración innecesaria y mantener el interés.

## Stakeholder Card

Owner / Equipo desarrollador

### Project:

Cat's Odyssey

### Stakeholder:

Owner / Equipo desarrollador

### Goal:

Completar el videojuego durante el semestre con una base de código que dos integrantes puedan entender, probar, corregir y ampliar sin reescribir grandes partes.

### Quality Attributes:

Modifiability (Maintainability) - QA-Priority: 3

Agregar o ajustar niveles, enemigos y mecánicas debe requerir cambios localizados y reutilizar lógica común.

Testability (Maintainability) - QA-Priority: 2

Los sistemas principales deben poder probarse y depurarse por separado para localizar fallos con rapidez.

Performance efficiency (Performance) - QA-Priority: 2

Movimiento, colisiones, enemigos y objetos deben ejecutarse con fluidez suficiente para no alterar la jugabilidad.

## Concern Card

Concern-ID: 1

### Concern:

¿Cómo construir los niveles para reutilizar reglas comunes y agregar nuevos escenarios sin programar cada nivel completo desde cero?

### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

### Options:

1. Un World independiente por nivel

Cada nivel implementa por separado plataformas, enemigos, checkpoints y lógica.

Impactos QA: = | = | = | -- | - | +

2. Clase base de nivel + configuración en código

Una estructura común concentra reglas compartidas y cada escenario configura su contenido.

Impactos QA: + | + | + | ++ | ++ | +

3. Niveles definidos mediante datos/configuración externa

El contenido del nivel se describe en datos que una lógica común interpreta.

Impactos QA: = | + | + | ++ | + | =

### Concern Card

Concern-ID: 2

#### Concern:

Cuando el personaje muere, ¿cómo restaurar el checkpoint y conservar el progreso relevante sin generar estados contradictorios?

#### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

#### Options:

1. Guardar solo la posición

Se registra únicamente dónde reaparece el jugador; el resto se reconstruye como al inicio.

Impactos QA: - | -- | - | + | + | ++

2. Estado de nivel en memoria

Un objeto conserva checkpoint, coleccionables y elementos relevantes mientras la partida está abierta.

Impactos QA: + | ++ | ++ | + | ++ | +

3. Persistencia local del progreso

Checkpoint y progreso se almacenan en un archivo local para recuperarlos incluso tras cerrar el juego.

Impactos QA: + | ++ | ++ | - | + | =

### Concern Card

Concern-ID: 3

#### Concern:

¿Cómo organizar caminar, saltar, caer, adherirse/trepar y empujar objetos para mantener controles consistentes y evitar conflictos entre acciones?

#### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

#### Options:

1. Condicionales dentro de Player/World

Las entradas y estados se controlan con múltiples if/else repartidos en las clases.

Impactos QA: - | - | - | -- | - | +

2. Estados explícitos de movimiento

El Player usa estados como GROUND, JUMP, FALL, CLIMB y PUSH con transiciones definidas.

Impactos QA: ++ | ++ | + | ++ | ++ | +

3. Componentes de movimiento separados

Caminar, saltar, trepar y empujar se implementan como comportamientos reutilizables coordinados.

Impactos QA: + | + | + | ++ | ++ | =

### Concern Card

Concern-ID: 4

#### Concern:

¿Cómo indicar qué superficies y objetos son interactivos para que el jugador reconozca dónde puede trepar, empujar o activar elementos?

#### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

#### Options:

1. Sin señal especial

El jugador descubre por ensayo y error qué elementos permiten interacción.

Impactos QA: -- | - | - | + | = | ++

2. Lenguaje visual consistente

Color, textura, animación o silueta distinguen superficies y objetos interactivos.

Impactos QA: ++ | + | ++ | + | + | +

3. Mensajes contextuales

Aparecen indicaciones breves cuando el jugador se acerca a un elemento interactivo.

Impactos QA: ++ | + | + | = | + | =

### Concern Card

Concern-ID: 5

#### Concern:

¿Cómo organizar intro tipo cómic, juego activo, diálogos, muerte, retorno al checkpoint y final de nivel sin mezclar todas las transiciones?

#### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

#### Options:

1. Booleanos y if/else en un World

Cada estado se controla mediante banderas dentro de una misma clase principal.

Impactos QA: - | - | - | -- | - | +

2. Worlds/pantallas separados + estado compartido

Intro, nivel y final se separan, compartiendo solo la información necesaria.

Impactos QA: ++ | ++ | + | + | ++ | +

3. Administrador explícito de estados

Un gestor controla estados y transiciones válidas del juego.

Impactos QA: ++ | ++ | ++ | ++ | ++ | =

## Concern Card

Concern-ID: 6

### Concern:

¿Cómo implementar enemigos que el jugador debe esquivar para crear distintos patrones sin reescribir toda la lógica?

### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

### Options:

1. Cada enemigo programado desde cero

Movimiento, detección y reacción se implementan individualmente.

Impactos QA: = | = | = | -- | - | +

2. Clase base Enemy + herencia/polimorfismo

Una clase común define reglas compartidas y subclases especializan el comportamiento.

Impactos QA: + | + | + | ++ | ++ | +

3. Comportamientos intercambiables

Patrullar, perseguir o volar se implementan como módulos/estrategias reutilizables.

Impactos QA: + | + | + | ++ | ++ | =

## Concern Card

Concern-ID: 7

### Concern:

¿Cómo implementar puzzles con cajas y objetos para que puedan reutilizarse en varios niveles y reaccionen de forma predecible?

### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

### Options:

1. Lógica específica por puzzle

Cada puzzle contiene sus propias reglas y comprobaciones.

Impactos QA: = | = | = | -- | - | +

2. Objetos interactivos con interfaz común

Cajas, interruptores y puertas comparten contratos de interacción reutilizables.

Impactos QA: ++ | + | + | ++ | ++ | +

3. Sistema de eventos entre objetos

Los objetos emiten/reciben eventos para activar puertas, mecanismos u otros elementos.

Impactos QA: + | + | ++ | ++ | + | =

### Concern Card

Concern-ID: 8

#### Concern:

¿Cómo gestionar coleccionables/recuerdos para que no se dupliquen, se conserven cuando corresponda y puedan usarse narrativamente?

#### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

#### Options:

1. Variables independientes por nivel

Cada escenario controla sus coleccionables con variables propias.

Impactos QA: = | - | = | -- | - | +

2. Registro central de coleccionables

Cada recuerdo tiene un ID y un registro común controla cuáles fueron obtenidos.

Impactos QA: + | ++ | ++ | ++ | ++ | +

3. Registro persistente en archivo local

Los IDs obtenidos se guardan para conservarlos entre sesiones.

Impactos QA: + | ++ | ++ | + | + | =

### Concern Card

Concern-ID: 9

#### Concern:

¿Cómo presentar tutoriales, diálogos y pistas sin interrumpir demasiado la exploración ni dejar al jugador sin información?

#### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

#### Options:

1. Tutorial obligatorio extenso

El juego explica todas las mecánicas antes de comenzar.

Impactos QA: + | + | - | = | + | +

2. Enseñanza contextual progresiva

Las indicaciones aparecen cuando una mecánica se usa por primera vez.

Impactos QA: ++ | ++ | ++ | + | ++ | +

3. Descubrimiento sin instrucciones directas

El entorno y el diseño del nivel deben enseñar las reglas sin textos de ayuda.

Impactos QA: - | - | + | + | = | ++

## Concern Card

Concern-ID: 10

---

### Concern:

¿Cómo organizar los dos power-ups previstos para poder ajustar sus efectos o añadir otro sin llenar el Player de condiciones especiales?

### QA order:

Learnability | Recovery | Satisfaction | Modifiability | Testability | Performance

### Options:

1. Condiciones específicas en Player

Cada power-up agrega variables y if/else directamente al personaje.

Impactos QA: = | = | = | -- | - | +

2. Interfaz/clase común PowerUp

Los power-ups implementan una estructura común y aplican su efecto mediante métodos definidos.

Impactos QA: + | + | + | ++ | ++ | +

3. Efectos configurables/componibles

Los power-ups se construyen combinando efectos reutilizables con duración y parámetros.

Impactos QA: + | + | + | ++ | + | =

## Event Card

Event 1: La fecha de entrega se adelanta

---

### Title:

La fecha de entrega se adelanta

### Description:

La fecha final del proyecto se reduce en dos semanas. El equipo debe priorizar soluciones que puedan implementarse, probarse y corregirse con menos retrabajo.

### Consequences:

Owner: Modifiability +1 de prioridad (máx. 3) y Testability +1 (máx. 3).

## Event Card

### Event 2: Pruebas con usuarios detectan confusión

---

**Title:**

Pruebas con usuarios detectan confusión

**Description:**

Varios jugadores no comprenden una mecánica principal durante sus primeros minutos y cometen repetidamente el mismo error.

**Consequences:**

Jugador: Learnability +1 de prioridad (máx. 3) y Recovery +1 (máx. 3).

## Event Card

### Event 3: Se duplica el contenido planificado

---

**Title:**

Se duplica el contenido planificado

**Description:**

El proyecto debe incorporar aproximadamente el doble de niveles, enemigos u objetos respecto del alcance inicialmente considerado.

**Consequences:**

Owner: Modifiability +1 de prioridad (máx. 3).



## Event Card

### Event 4: Caídas de rendimiento en pruebas

---

**Title:**

Caídas de rendimiento en pruebas

**Description:**

Al aumentar simultáneamente objetos, enemigos y efectos, la fluidez baja lo suficiente como para afectar controles y tiempos de reacción.

**Consequences:**

Owner: Performance efficiency +1 de prioridad (máx. 3). Jugador: Satisfaction +1 (máx. 3).

## Event Card

### Event 5: Los errores bloquean el progreso

---

**Title:**

Los errores bloquean el progreso

**Description:**

Durante pruebas aparecen fallos que dejan al jugador en estados donde no puede continuar o recuperar correctamente su progreso.

**Consequences:**

Jugador: Recovery +1 de prioridad (máx. 3). Owner: Testability +1 (máx. 3).